



Projektdokumentation

in Phase 3 aktualisiert

Projekttitel: Network Architect

Kurs: Software Engineering (DLMCSPSE01_D)

Student: Johannes Öllerer

Matrikelnummer: 92211477

Datum: 01.August 2026



Inhaltsverzeichnis

1 Projektübersicht.....	4
1.1 Projektbeschreibung.....	4
1.2 Problemstellung und Zielhypothese.....	4
1.3 Angestrebtes Ergebnis	5
1.4 Zielgruppe	6
1.5 Abgrenzung zu existierenden Lösungen.....	7
1.6 Limitationen	7
1.7 GitHub Repository Link	8
2 Vorgehensmodell	9
2.1 Wahl des Vorgehensmodells: Iteratives Wasserfallmodell	9
2.2 Diagramm des iterativen Wasserfallmodells.....	10
2.3 Ausgestaltung der Projektphasen im iterativen Wasserfallmodell.....	10
3 Risikomanagement.....	12
3.1 Risikoübersicht	12
3.2 Risikobeschreibung.....	12
3.2.1 TECHNISCHE RISIKEN.....	12
3.2.2 ZEITLICHE RISIKEN.....	15
3.2.3 SCOPE/ANFORDERUNGS-RISIKEN	16
3.2.4 EXTERNE ABHÄNGIGKEITEN.....	16
3.2.5 Sicherheitsrisiken.....	17
3.3 Risiko-Priorisierung.....	18
4 Zeitplanung.....	19
4.1 Gantt-Diagramm	19
4.2 Arbeitspakete.....	20
4.3 Detailbeschreibung und Meilensteine	23
4.4 Plan-Ist-Vergleich der Zeitplanung.....	25
4.5 Reflexion der Zeitplanung und Abweichungen.....	28
5 Benutzeranleitung	28

5.1 Installation und Start.....	28
5.2 Kurzanleitung	29
6 Reflexion zentraler Entscheidungen und Alternativen.....	31
6.1 Reflexion des Vorgehensmodells.....	31
6.2 Reflexion technologischer Entscheidungen	31
6.3 Reflexion der Architekturentscheidung.....	33
6.4 Betrachtete Alternativen	33
7 Zusammenfassung und Ausblick.....	35
8 Verzeichnisse.....	36
8.1 Tabellenverzeichnis	36
8.2 Abbildungsverzeichnis.....	36
8.3 Abkürzungsverzeichnis	36

1 Projektübersicht

1.1 Projektbeschreibung

Network Architect

revolutioniert das klassische Hashiwokakero-Spielprinzip durch eine moderne IT-Thematik und erweiterte Spielmechaniken. Die Anwendung verbindet bewährte Puzzle-Logik mit zeitgemäßen Netzwerk-Konzepten und schafft dadurch ein einzigartiges Spielerlebnis für Puzzle-Enthusiasten.

Kernkonzept

Traditionelle "Inseln" und "Brücken" werden zu Netzwerkknoten (Server, Router, Switches) und Datenverbindungen (Ethernet, Glasfaser, WLAN). Spieler entwerfen funktionsfähige IT-Infrastrukturen unter Berücksichtigung realer Netzwerk-Prinzipien wie Bandbreite, Latenz und Redundanz.

Spielmechanik

Network Architect behält die klassischen Hashiwokakero-Regeln bei, wobei Zahlen auf Netzwerkknoten die Anzahl erforderlicher Verbindungen definieren. Die Innovation liegt in verschiedenen Verbindungstypen (Ethernet, Glasfaser, WLAN) mit unterschiedlichen Performance-Eigenschaften wie Bandbreite und Latenz. Spieler müssen nicht nur gültige, sondern auch optimierte Netzwerk-Topologien erstellen und Redundanz für Ausfallsicherheit berücksichtigen.

Progressive Komplexität führt von einfachen LAN-Setups zu mehrschichtigen WAN-Infrastrukturen mit verschiedenen Netzwerk-Protokollen. Echtzeit-Performance-Feedback bewertet die Effizienz der Lösung, während ein Achievement-System besonders elegante und innovative Netzwerk-Designs belohnt. Diese Kombination aus klassischer Puzzle-Logik und authentischen IT-Prinzipien schafft strategische Tiefe über das reine Brücken-Bauen hinaus.

1.2 Problemstellung und Zielhypothese

Klassische Logikspiele wie Hashiwokakero bieten zwar anspruchsvolle Rätsel, bleiben jedoch weitgehend abstrakt und vermitteln kaum übertragbares Wissen zu realen IT-Domänen. Gleichzeitig werden Netzwerk-Konzepte in Ausbildung und Studium häufig theoretisch oder in komplexen Laborumgebungen vermittelt, was den Einstieg für Lernende erschwert. Es fehlt somit an einem niedrigschwelligen, spielerischen Format,

das logische Herausforderung mit einem verständlichen Zugang zu grundlegenden Netzwerk-Prinzipien verbindet.

Die daraus resultierende Problemstellung lautet: Wie kann ein Desktop-Puzzle-Spiel gestaltet werden, das die bewährten Regeln von Hashiwokakero beibehält und zugleich reale Netzwerk-Prinzipien wie Bandbreite, Latenz und Redundanz so integriert, dass sowohl Puzzle-Enthusiasten als auch IT-Studierende eine verständliche und motivierende Spielerfahrung erhalten?

Die Zielhypothese von Network Architect ist, dass die Kombination des klassischen Hashiwokakero-Prinzips mit einer realitätsnahen Netzwerk-Thematik ein Puzzle-Spiel ermöglicht, das einerseits zusätzliche strategische Tiefe durch Performance- und Redundanzbewertung schafft und andererseits ein intuitiveres Verständnis grundlegender Netzwerk-Konzepte fördert. Das Projekt untersucht zudem, wie sich etablierte Software-Engineering-Methoden im Rahmen eines iterativen Wasserfallmodells auf die Entwicklung eines edukativen Puzzlespiels anwenden lassen

1.3 Angestrebtes Ergebnis

Das angestrebte Ergebnis ist eine vollständig funktionsfähige Desktop-Anwendung unter Windows 10/11, die als innovative Weiterentwicklung des klassischen Hashiwokakero-Spiels fungiert. Network Architect soll eine Software darstellen, die Puzzle-Enthusiasten durch moderne IT-Thematik eine frische Herausforderung bietet.

Die finale Anwendung soll durch die Implementierung authentischer Netzwerk-Prinzipien und Performance-Feedback ein einzigartiges Spielerlebnis schaffen, das sowohl die logischen Fähigkeiten schärft als auch ein intuitives Verständnis für IT-Infrastrukturen vermittelt. Das Endprodukt repräsentiert eine erfolgreiche Symbiose aus bewährter Puzzle-Tradition und zeitgemäßer Technologie-Thematik, die neue Maßstäbe im Puzzle-Gaming-Bereich setzt. Zusammengefasst ist das angestrebte Ergebnis eine funktionsfähige Desktop-Anwendung, die folgende Eigenschaften aufweist:

- Logik-Puzzle-Gameplay mit Hashiwokakero-Regeln bietet
- Realistische Netzwerk-Simulation mit Performance-Metriken implementiert
- Educational Value durch echte IT-Konzepte vermittelt
- Interaktive GUI mit Drag&Drop und visuellen Feedback-Systemen bietet
- Multiple Level mit steigendem Schwierigkeitsgrad enthält

1.4 Zielgruppe

PRIMÄRE ZIELGRUPPE: Puzzle-Enthusiasten

- Nutzen: Genießen logische Herausforderungen mit modernem Twist
- Bedarf: Innovative, anspruchsvolle Rätsel mit zeitgemäßen Themen
- Customer Value: Erfahrene Puzzle-Spieler erhalten eine frische Interpretation des klassischen Hashiwokakero-Prinzips. Das moderne IT-Theme bietet neue strategische Dimensionen und Performance-Aspekte, die über einfaches Brücken-Bauen hinausgehen.

SEKUNDÄRE ZIELGRUPPE: IT-Studenten

- Nutzen: Lernen Netzwerk-Design-Prinzipien spielerisch
- Bedarf: Praktische Anwendung theoretischer IT-Kenntnisse in einer risikofreien Umgebung
- Customer Value: IT-Studenten entwickeln durch den spielerischen Ansatz ein intuitiveres Verständnis für Netzwerk-Grundlagen und entdecken Interesse an Infrastruktur-Design. Die Anwendung macht abstrakte Netzwerk-Prinzipien greifbar und motiviert zur weiteren Beschäftigung mit IT-Themen durch unterhaltsame Wissensvermittlung.

1.5 Abgrenzung zu existierenden Lösungen

Standard Hashiwokakero-Spiele:

- ✗ Abstrakte Spielelemente ohne Realitätsbezug
- ✗ Reine Logik-Rätsel ohne praktischen Lernwert
- ✗ Statische Win/Lose-Bewertung

Traditionelle Hashiwokakero-Spiele verwenden abstrakte Inseln und Brücken ohne Realitätsbezug und funktionieren als reine Logik-Rätsel ohne praktischen Lernwert. Die Bewertung erfolgt nach einem simplen Win/Lose-Schema ohne Optimierungsmöglichkeiten oder qualitative Abstufungen.

Unser Network Architect:

- ✓ Realistische IT-Infrastruktur-Simulation
- ✓ Performance-Metriken (Latenz, Bandwidth, Kosten)
- ✓ Educational Gameplay mit echten Netzwerk-Konzepten
- ✓ Dynamic Feedback und Optimierungsvorschläge

Network Architect simuliert realistische IT-Infrastrukturen mit authentischen Netzwerk-Komponenten und vermittelt echte IT-Konzepte durch Educational Gameplay. Performance-Metriken wie Latenz und Bandwidth ermöglichen nuancierte Lösungsoptimierung, während dynamisches Feedback kontinuierliches Lernen und Verbesserung unterstützt.

1.6 Limitationen

Die Kapitel 1.1 bis 1.5 beschreiben Projektidee, angestrebtes Ergebnis, Zielgruppen und die Abgrenzung zu bestehenden Lösungen. Trotz dieses klar definierten Scopes weist Network Architect mehrere bewusste Einschränkungen auf, die sich aus dem vorgegebenen Projektumfang und den Rahmenbedingungen der Prüfungsleistung ergeben.

Fachliche Vereinfachungen

Fachlich bildet Network Architect nur ausgewählte, bewusst vereinfachte Aspekte realer IT-Netzwerke ab. Die Anwendung modelliert keine konkreten Netzwerk-Protokolle oder

Routing-Algorithmen und berechnet auch keine realistischen Latenz- oder Bandbreitenwerte, sondern verwendet abstrahierte Metriken zur Bewertung von Performance und Redundanz. Darüber hinaus konzentriert sich das Spiel ausschließlich auf Einzelspieler-Puzzles; kooperative Szenarien, dynamische Ereignisse wie Server-Ausfälle zur Laufzeit oder komplexe Kampagnenstrukturen wurden nicht umgesetzt.

Technische Einschränkungen

Technisch ist Network Architect explizit als Desktop-Anwendung für Windows 10/11 ausgelegt; andere Betriebssysteme wurden im Rahmen des Projekts weder getestet noch offiziell unterstützt. Die Persistenz beruht auf einer lokalen SQLite-Datenbank, wodurch Mehrbenutzer-Szenarien und verteilte Anwendungen bewusst ausgeschlossen sind. Die grafische Benutzeroberfläche orientiert sich an einer klassischen Maus-und-Tastatur-Interaktion; Aspekte wie Barrierefreiheit, alternative Eingabemethoden oder Touch-Optimierung wurden aus Umfangsgründen nicht systematisch adressiert.

Zusätzlich wurde im Projektverlauf als technische Schuld identifiziert, dass die Prüfzeit für Lösungs- und Redundanzprüfungen bei großen Levels mit vielen Bridges deutlich ansteigt. Eine grundlegende algorithmische Optimierung dieser Prüfmechanismen war im gegebenen zeitlichen und inhaltlichen Rahmen des Projekts nicht umsetzbar, da der Fokus auf der funktionalen Fertigstellung und Dokumentation der Kernanforderungen lag. Diese Einschränkung ist im Testdokument, Kapitel 4.4 Nichtfunktionale Tests, referenziert.

Methodische Grenzen

Auch methodisch ist das Projekt durch mehrere Grenzen geprägt. Es wurden keine formalen Usability-Tests mit repräsentativen Nutzergruppen durchgeführt; Rückmeldungen zur Bedienbarkeit basieren vor allem auf Eigenbeurteilung und Tutor-Feedback. Die automatisierten Tests fokussieren auf die zentrale Spiel- und Netzwerklogik, während GUI-Randfälle und Fehlerszenarien nur teilweise durch Tests abgedeckt sind. Darüber hinaus wurde das iterative Wasserfallmodell im Rahmen eines Einzelprojekts mit begrenzten Iterationsschleifen angewendet, sodass keine Aussagen über seine Wirksamkeit in größeren Team-Setups getroffen werden können.

1.7 GitHub Repository Link

<https://github.com/besidesmax/NetworkArchitect.git>

2 Vorgehensmodell

2.1 Wahl des Vorgehensmodells: Iteratives Wasserfallmodell

Für die Durchführung des Softwareprojekts wurde das iterative Wasserfallmodell gewählt. Diese Entscheidung basiert auf der Analyse der Rahmenbedingungen des Portfolios (=Aufgabenstellung), welches eine klare Strukturierung in die Phasen Konzeption, Realisierung und Abschluss vorgibt.

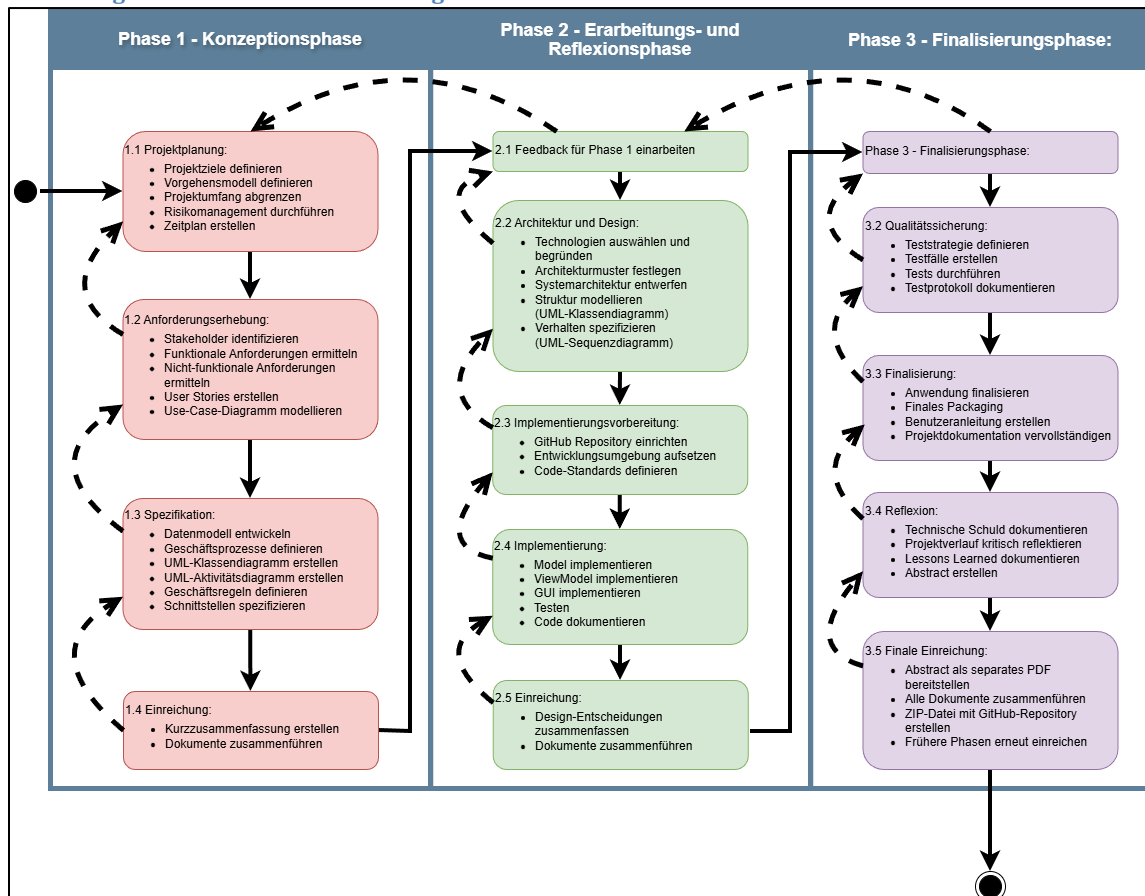
Im Gegensatz zum klassischen Wasserfallmodell erlaubt der gewählte iterative Ansatz explizite Rückkopplungsschleifen (Feedback Loops) zwischen den Phasen. Diese sind auch explizit in der Aufgabenstellung gefordert. Dies ist notwendig, da Softwareentwicklung ein erkenntnisgetriebener Prozess ist. Erkenntnisse, die erst während der Implementierung (Phase 2) oder dem Testing (Phase 3) gewonnen werden, führen zu einer kontrollierten Überarbeitung der Artefakte früherer Phasen (z. B. Anpassung der Spezifikation oder Architektur).

Vorteile für dieses Projekt:

- Konformität: Die sequenzielle Grundstruktur entspricht den Abgabeterminen der Prüfungsleistung.
- Risikominimierung: Fehler in der Anforderungsanalyse werden nicht „mitgeschleppt“, sondern durch iterative Verfeinerung korrigiert.
- Konsistenz: Code und Dokumentation bleiben durch die Rücksprünge synchron.

2.2 Diagramm des iterativen Wasserfallmodells

Abbildung 1: Iteratives Wasserfall-Diagramm



Quelle: Erstellt mit draw.io

2.3 Ausgestaltung der Projektphasen im iterativen Wasserfallmodell

In Phase 1 (Planung und Anforderungsanalyse) werden die Grundlagen des Projekts gelegt: Zunächst erfolgt die Projektplanung mit Zieldefinition, Risikobetrachtung und Zeitplan, anschließend werden funktionale und nicht-funktionale Anforderungen erhoben und als User Stories sowie in einem UML-Use-Case-Diagramm dokumentiert. Auf Basis dieser Anforderungen entsteht die fachliche Spezifikation mit Datenmodell, Geschäftsprozessen, Geschäftsregeln sowie einer Beschreibung der wichtigsten Benutzeroberflächen. Die in dieser Phase erstellten Artefakte bilden die Referenz für alle weiteren Aktivitäten und werden in der Projektdokumentation, dem Anforderungsdokument und dem Spezifikationsdokument festgehalten.

Phase 2 (Erarbeitungs- und Reflexionsphase) übernimmt diese Ergebnisse und überführt sie in eine konkrete technische Lösung. Dazu wird zunächst ein Architekturdokument erstellt, das die gewählte Technologie, das Architekturkonzept sowie die zentralen Klassen und ihre Beziehungen in Form eines UML-Klassendiagramms und eines Sequenzdiagramms beschreibt. Auf dieser Basis wird die Anwendung implementiert, im GitHub-Repository versioniert und laufend mit den Anforderungen und Spezifikationen abgeglichen; notwendige Anpassungen werden dabei dokumentiert und begründet.

In Phase 3 (Test und Abschluss) liegt der Fokus auf der Qualitätssicherung und dem geordneten Projektabschluss. Es werden Teststrategie und Testfälle definiert, Tests durchgeführt und deren Ergebnisse in einem Testdokument protokolliert, bevor die Anwendung finalisiert und um eine Benutzeranleitung ergänzt wird. Abschließend wird das Projekt in einem zweiseitigen Abstract reflektiert, der zentrale Entscheidungen, Abweichungen vom ursprünglichen Plan und Lessons Learned zusammenfasst.

Die Iterationen zwischen den Phasen sind bewusst entlang der Vorgaben der Aufgabenstellung geplant, die explizite Feedbackschleifen zwischen Konzeptions-, Erarbeitungs- und Test und Abschluss vorsieht. Typische Rücksprünge im Projekt „Network Architect“ entstehen beispielsweise, wenn während der Implementierung des Spielregelsystems oder der MVVM-Struktur Unklarheiten in einer User Story auffallen und dadurch eine Präzisierung der Anforderungen oder eine Anpassung des Datenmodells notwendig wird. Ebenso können Testergebnisse in Phase 3, etwa fehlerhafte Berechnungen der Brückenanzahl oder unklare GUI-Abläufe, gezielt zu Überarbeitungen von Sequenzdiagrammen, Architekturentscheidungen oder Spezifikation führen, wobei jede Änderung dokumentiert und – falls sie nicht mehr umgesetzt werden kann – als technische Schuld festgehalten wird.

3 Risikomanagement

3.1 Risikoübersicht

Tabelle 1: Risikoübersicht

Risiko	Eintrittswahrscheinlichkeit	Auswirkung	Erwartungswert	Risiko-Level	Priorität
R1: Dependency-Kompatibilität	 Hoch	 Hoch	9	 Hoch	2
R2: Performance	 Mittel	 Mittel	4	 Mittel	6
R3: DB-Korruption	 Niedrig	 Hoch	3	 Mittel	7
R4: UI-Komplexität	 Hoch	 Mittel	6	 Hoch	3
R5: Learning Curve	 Mittel	 Mittel	4	 Mittel	8
R6: Feature-Creep	 Mittel	 Hoch	6	 Hoch	4
R7: GitHub-Ausfall	 Niedrig	 Hoch	3	 Mittel	9
S1: SQLite-Injection-Vulnerabilities	 Hoch	 Kritisch	12	 Hoch	1
S2: Unsichere Deserialisierung bei Level-Import	 Mittel	 Hoch	6	 Hoch	5

Quelle: Eigene Darstellung

3.2 Risikobeschreibung

3.2.1 TECHNISCHE RISIKEN

R1: Dependency-Kompatibilität

- Beschreibung: Die Python-Anwendung Network Architect ist auf komplexe Dependencies (Python Runtime, PySide6/Qt6, System-Bibliotheken) angewiesen, deren Kompatibilität auf verschiedenen Windows-Zielsystemen kritisch für den Projekterfolg ist. Die hohe Eintrittswahrscheinlichkeit resultiert aus der bekannten Problematik von Python-Desktop-Deployment, besonders bei PySide6-Anwendungen, die erst bei finalen PyInstaller-Tests auf fremden Systemen auftreten. Ein Versagen der Dependency-Kompatibilität würde das komplette Projekt gefährden, da das Resultat eine nicht-funktionsfähige Anwendung zur Folge hat.
- Wahrscheinlichkeit: HOCH | Auswirkung: HOCH | Level: HOCH

- Gegenmaßnahmen: Implementierung einer robusten PyInstaller-Konfiguration mit expliziter Definition aller Hidden-Imports und Asset-Dependencies zur vollständigen Erfassung benötigter Module. Einsatz spezialisierter Analyse-Tools zur präventiven Prüfung der finalen Anwendungs-Bundles vor Distribution. Multi-Environment-Testing auf verschiedenen Windows-Konfigurationen zur frühzeitigen Identifikation von Kompatibilitätsproblemen.
- Frühwarnindikatoren: Wiederholte Fehler beim Erzeugen von PyInstaller-Builds oder abweichendes Verhalten der Anwendung auf unterschiedlichen Testsystemen.

R2: Performance bei komplexen Netzwerk-Visualisierungen

- Beschreibung: QGraphicsView-basierte Netzwerk-Visualisierungen können bei größeren Puzzle-Layouts mit 100+ Netzwerk-Knoten und entsprechenden Verbindungen zu Performance-Problemen führen, die sich in träger Benutzerinteraktion und verzögertem Rendering äußern. Das Risiko steigt mit der Komplexität der dargestellten Netzwerk-Topologien, da jeder Knoten und jede Verbindung als separates QGraphicsItem gerendert wird.
- Wahrscheinlichkeit: MITTEL | Auswirkung: MITTEL | Level: MITTEL
- Gegenmaßnahmen: Implementierung bewährter QGraphicsView-Optimierungen durch Setzen von Optimierungsflags. Ergänzende Performance-Profiling ab 50+ Netzwerk-Knoten zur frühzeitigen Identifikation von Engpässen. Optionaler Einsatz von Worker-Threads zur Entkopplung von Level-Berechnungen vom Haupt-UI-Thread. Diese bewährten Qt-Best-Practices reduzieren die Eintrittswahrscheinlichkeit von Performance-Problemen auf Hashiwokakero-Level (10-100 Knoten) signifikant von Mittel auf Niedrig.
- Frühwarnindikatoren: Spürbare Verzögerungen beim Laden größerer Level oder beim Validieren der Lösung auf dem Entwicklerrechner.

R3: SQLite-Datenbankkorruption

- Beschreibung: Unerwartete Programmbeendigung durch Systemabstürze, Stromausfälle oder Anwendungs-Crashes können zur Beschädigung der SQLite-Datenbank führen, die sämtliche Spielstände, Level-Definitionen und Benutzereinstellungen enthält.
- Wahrscheinlichkeit: NIEDRIG | Auswirkung: HOCH | Level: MITTEL
- Gegenmaßnahmen: Implementierung transaktionaler Speicheroperationen mit automatischem Rollback bei fehlerhaften Schreibvorgängen, regelmäßige automatische Backup-Erstellung der Datenbankdateien sowie

Datenbankintegritäts-Checks beim Anwendungsstart zur frühzeitigen Erkennung von Korruptionen. Diese defensive Programmierung gewährleistet Datenintegrität und ermöglicht eine schnelle Wiederherstellung im Schadensfall.

- Frühwarnindikatoren: Gelegentliche Fehlermeldungen beim Lesen/Schreiben der SQLite-Datenbank oder unerwartete Inkonsistenzen in gespeicherten Spielständen.

3.2.2 ZEITLICHE RISIKEN

R4: Unterschätzung der UI-Komplexität

- Beschreibung: Die Entwicklung einer intuitiven Drag&Drop-Benutzeroberfläche mit Echtzeit-Performance-Feedback und visuellen Netzwerk-Darstellungen erfordert erheblich mehr Entwicklungszeit als initial veranschlagt, da komplexe Interaktionsmuster und responsive UI-Elemente aufwändige Implementierung und umfangreiche Testphasen benötigen. Die Unterschätzung des UI-Entwicklungsaufwands kann zu erheblichen Zeitverzögerungen führen und andere Projektphasen gefährden.
- Wahrscheinlichkeit: HOCH | Auswirkung: MITTEL | Level: NIEDRIG
- Gegenmaßnahmen: Konsequente Priorisierung von Core-Features vor Advanced-UI-Elementen, iterative UI-Entwicklung mit funktionsfähigen Minimal-Prototypen zur frühzeitigen Validierung von Interaktionskonzepten sowie die Einplanung ausreichender Buffer-Zeit für UI-Polish und Usability-Optimierungen.
- Frühwarnindikatoren: Basis-GUI mit Spielfeld, Menüführung und Drag-&-Drop ist bis Ende Woche 4 nicht voll funktionsfähig oder enthält blockierende Usability-Probleme.

R5: Verspätung durch Learning Curve

- Beschreibung: Die Einarbeitung in neue Technologien wie QGraphicsView für komplexe 2D-Visualisierungen und das MVVM-Architekturpattern für Desktop-Anwendungen kann erheblich mehr Zeit beanspruchen als geplant, da diese Konzepte praktische Erfahrung und experimentelle Implementierung erfordern. Unvorhergesehene technische Herausforderungen bei der Integration von Canvas-basierten Interaktionen mit reaktivem Data-Binding können zu Projektverzögerungen führen.
- Wahrscheinlichkeit: MITTEL | Auswirkung: MITTEL | Level: MITTEL
- Gegenmaßnahmen: Frühzeitige Proof-of-Concept-Implementierung der kritischsten Technologie-Komponenten, paralleles Learning während der Entwicklung einfacherer Features sowie die Vorbereitung von Fallback-Lösungen mit simpleren UI-Ansätzen falls die gewünschte Technologie-Integration zu zeitaufwändig wird.
- Frühwarnindikatoren: Geplante Kernfeatures (z. B. MVVM-Binding, QGraphicsView-Canvas) bleiben über mehrere Wochen im Status „Prototyp“ ohne stabilen Fortschritt.

3.2.3 SCOPE/ANFORDERUNGS-RISIKEN

R6: Feature-Creep bei Netzwerk-Simulation

- Beschreibung: Die Verlockung, das IT-Netzwerk-Theme durch immer detailliertere und authentischere Netzwerk-Features zu erweitern, kann dazu führen, dass die ursprüngliche Puzzle-Mechanik in den Hintergrund gedrängt wird und das Spiel zu komplex oder unspielbar wird. Übermäßige Fokussierung auf realistische IT-Konzepte gefährdet das fundamentale Hashiwokakero-Gameplay und kann die primäre Zielgruppe der Puzzle-Enthusiasten abschrecken.
- Wahrscheinlichkeit: MITTEL | Auswirkung: HOCH | Level: HOCH
- Gegenmaßnahmen: Definition einer klaren MVP-Version mit ausschließlich Must-Have-Features, einen strikten Feature-Freeze nach der Planung und Anforderungsanalyse sowie regelmäßige Scope-Reviews gegen die ursprüngliche Aufgabenstellung zur Sicherstellung der Projekt-Fokussierung.
- Frühwarnindikatoren: Zunehmende Anzahl neuer Feature-Ideen im Backlog, während Must-have-Aufgaben regelmäßig verschoben werden.

3.2.4 EXTERNE ABHÄNGIGKEITEN

R7: GitHub-Verfügbarkeit

- Beschreibung: Ein Ausfall der GitHub-Plattform während der aktiven Entwicklungsphase oder unmittelbar vor der Projekt-Deadline könnte den Zugriff auf das zentrale Repository mit Versionshistorie, Quellcode und Projektdokumentation verhindern und damit sowohl den Entwicklungsfortschritt als auch die rechtzeitige Abgabe gefährden. Besonders kritisch wäre ein Ausfall in den finalen Projekttagen, wenn intensive Entwicklungsarbeit und Dokumentationserstellung stattfindet.
- Wahrscheinlichkeit: NIEDRIG | Auswirkung: HOCH | Level: MITTEL
- Gegenmaßnahmen: Erstellung regelmäßiger lokaler Git-Backups zusätzlich zum GitHub-Repository, frühzeitige Abgabe-Vorbereitung mit mindestens 48-Stunden-Zeitpuffer vor Deadline. Diese redundante Backup-Strategie gewährleistet kontinuierliche Entwicklungsarbeit und rechtzeitige Projekt-Abgabe unabhängig von der Verfügbarkeit der GitHub-Plattform.
- Frühwarnindikatoren: Wiederholte Zugriffsprobleme auf GitHub oder fehlende aktuelle lokale Backups des Repositories über mehrere Tage.

3.2.5 Sicherheitsrisiken

S1: SQLite-Injection-Vulnerabilities

- Beschreibung: Benutzereingaben wie Level-Namen, Spieler-Namen oder Konfigurationsparameter werden ohne ausreichende Validierung oder Parameterisierung direkt in SQLite-Datenbankabfragen eingebettet, wodurch Angreifer durch manipulierte Eingaben SQL-Befehle einschleusen können. Dies ermöglicht unbefugten Zugriff auf die Spielstand-Datenbank, Manipulation von Level-Daten oder komplette Datenbankzerstörung durch DROP-Statements, was sowohl Datenintegrität als auch Anwendungsstabilität gefährdet.
- Wahrscheinlichkeit: HOCH | Auswirkung: KRITISCH | Level: HOCH
- Gegenmaßnahmen: Konsequente Verwendung von Prepared Statements für alle Datenbankoperationen, umfassende Input-Validierung und Sanitization aller Benutzereingaben sowie Implementierung des Principle of Least Privilege für Datenbankzugriffe mit minimalen erforderlichen Berechtigungen.
- Frühwarnindikatoren: Sichtbare String-Konkatenation in SQL-Statements ohne Parameterbindung oder fehlende Tests zur Eingabvalidierung in datenbanknahen Komponenten.

S2: Unsichere Deserialisierung bei Level-Import

- Beschreibung: Das Laden von benutzerdefinierten Level-Dateien oder Spielständen ohne ausreichende Validierung ermöglicht die Einschleusung von schadhaftem Code. Manipulierte Level-Dateien können beliebigen Python-Code zur Ausführung bringen und damit vollständige Systemkontrolle erlangen, Malware installieren oder sensitive Daten kompromittieren, was bei Level-Sharing-Features oder Import-Funktionalitäten zu kritischen Sicherheitslücken führt.
- Wahrscheinlichkeit: MITTEL | Auswirkung: HOCH | Level: HOCH
- Gegenmaßnahmen: Ausschließliche Verwendung sicherer Serialisierungsformate wie JSON oder XML anstatt Python-Pickle, Implementierung strikter Schema-Validierung für alle importierten Dateien sowie Sandbox-Umgebung für das Laden externer Level-Daten mit eingeschränkten Systemzugriffen.
- Frühwarnindikatoren: Einsatz unsicherer Deserialisierungsmechanismen in Entwürfen oder Implementierungen sowie fehlende Schema-Validierung für Importdateien.

3.3 Risiko-Priorisierung

Zur Priorisierung der identifizierten Risiken wird ein Erwartungswert verwendet, der die Kombination aus Eintrittswahrscheinlichkeit und Schadenshöhe quantitativ abbildet. Die Eintrittswahrscheinlichkeit kann die Stufen „niedrig“, „mittel“ und „hoch“ annehmen und wird numerisch wie folgt hinterlegt:

- Niedrig = 1
- Mittel = 2
- Hoch = 3

Die Schadenshöhe (Auswirkung) kann die Stufen „niedrig“, „mittel“, „hoch“ und „kritisch“ annehmen und wird wie folgt skaliert:

- Niedrig = 1
- Mittel = 2
- Hoch = 3
- Kritisch = 4

Für jedes Risiko wird der Erwartungswert als Produkt der beiden Skalenwerte berechnet:

$EW = \text{Wahrscheinlichkeit} \times \text{Schadenshöhe}$

Ein hoher Erwartungswert signalisiert eine hohe Priorität, da entweder die Eintrittswahrscheinlichkeit, die Schadenshöhe oder beide Dimensionen groß sind. Risiken mit hohem Erwartungswert werden frühzeitig in Phase 2 adressiert, während Risiken mit mittlerem Erwartungswert flankierend und über die Projektlaufzeit verteilt behandelt werden.

Risiken mit besonders hoher Priorität sind R1 (Dependency-Kompatibilität), R4 (UI-Komplexität), R6 (Feature-Creep), S1 (SQL-Injection, kritisch) und S2 (unsichere Deserialisierung, hoch). Diese Risiken werden in der Implementierungsplanung und Architekturgestaltung gezielt berücksichtigt, etwa durch frühe Absicherung der Build-Umgebung, Security-Best-Practices und klare Scope-Grenzen. Risiken mit mittlerem Erwartungswert wie R2 (Performance), R3 (DB-Korruption), R5 (Learning Curve) und R7 (GitHub-Verfügbarkeit) werden über bewährte Vorgehensweisen, Backup-Strategien und kontinuierliche Optimierung kontrolliert.

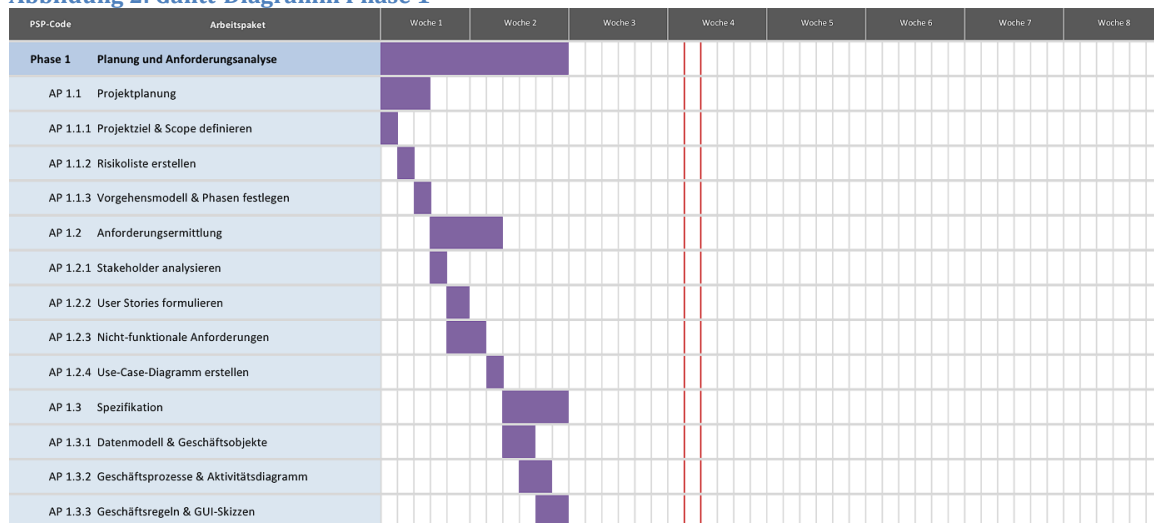
4 Zeitplanung

Dieses Kapitel dokumentiert die Zeitplanung des Projekts Network Architect gemäß iterativem Wasserfallmodell. Es umfasst geplante Arbeitspakete, Gantt-Diagramme und den Plan-Ist-Vergleich zur Reflexion von Abweichungen.

4.1 Gantt-Diagramm

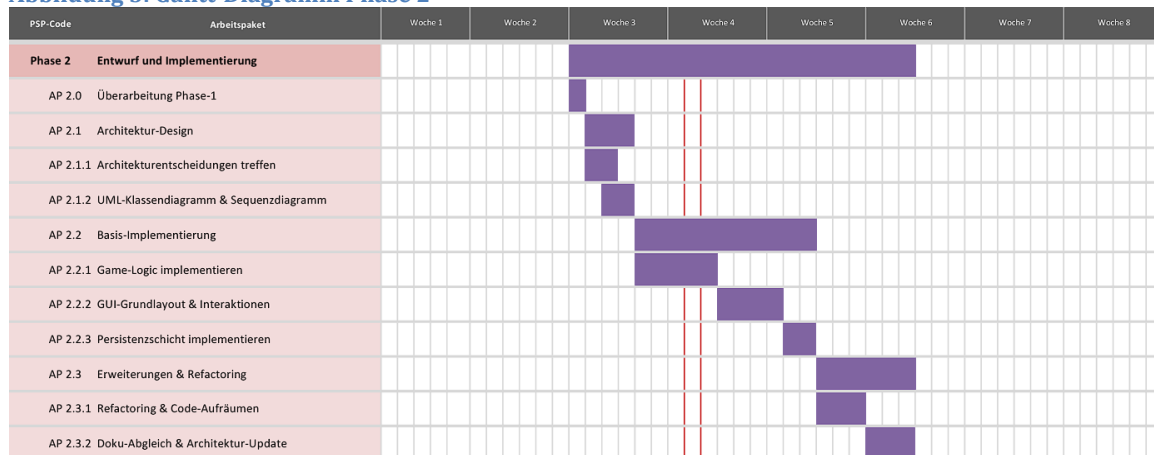
Das Diagramm zeigt die zeitliche Strukturierung des Projekts in drei Phasen über acht Wochen mit zentralen Meilensteinen. **Abbildung 2** visualisiert die Phase „Planung und Anforderungsanalyse“ (Wochen 1–2), **Abbildung 3** die Phase „Entwurf und Implementierung“ (Wochen 3–6) und **Abbildung 4** die Phase „Test und Abschluss“ (Wochen 7–8). In der Phase „Planung und Anforderungsanalyse“ (Abbildung 2) wurden Projektplanung, Anforderungsermittlung und Spezifikation mit insgesamt 40 Stunden veranschlagt (Meilensteine M1–M3), während die Phase „Entwurf und Implementierung“ (Abbildung 3) mit 77–87 Stunden den größten Aufwand umfasst (Meilensteine M4–M6). Die Phase „Test und Abschluss“ (Abbildung 4) bündelt Testdurchführung und Projektabschluss mit insgesamt 45 Stunden (Meilensteine M7–M8)

Abbildung 2: Gantt-Diagramm Phase 1



Quelle: Erstellt mit Excel

Abbildung 3: Gantt-Diagramm Phase 2



Quelle: Erstellt mit Excel

Abbildung 4: Gantt-Diagramm Phase 3



Quelle: Erstellt mit Excel

4.2 Arbeitspakete

Die zeitliche Planung des Projekts erfolgt auf Basis von Arbeitspaketen, die den drei Projektphasen zugeordnet sind und jeweils Aufwand, Zeitraum und zugehörige Meilensteine präzisieren. Tabelle 2 zeigt die hierarchische Gliederung des Projektstrukturplans (PSP-Codes), die geplanten Kalenderwochen sowie die geschätzten Stunden pro Arbeitspaket. Die in Tabelle 2 aufgeführten Arbeitspakete bilden die Grundlage für das Gantt-Diagramm.

Tabelle 2: Arbeitspakete

PSP-Code	Arbeitspaket	Kurzbeschreibung	Start (Wo)	Ende (Wo)	Aufwand (h)	Meilenstein
1	Planung und Anforderungsanalyse	Gem. Aufgabenstellung	Wo 1	Wo 2	40	M1-M3
1.1	Projektplanung	Projektziel, Risiken, Vorgehensmodell und Zeitplan definieren	Wo 1	Wo 1	8	M1
1.1.1	Projektziel & Scope definieren	Problemabgrenzung, Zielbild und Kundennutzen beschreiben	Wo 1	Wo 1	3	

1.1.2	Risikoliste erstellen	Risiken identifizieren, bewerten, Gegenmaßnahmen grob planen	Wo 1	Wo 1	3	
1.1.3	Vorgehensmodell & Phasen festlegen	„Vorgehensmodell auswählen und begründen; Rollen und Ergebnisse je Projektphase festlegen.“	Wo 1	Wo 1	2	
1.2	Anforderungs- ermittlung	Stakeholder, User Stories, funktionale/nicht-funktionale ANF	Wo 1	Wo 2	15	M2
1.2.1	Stakeholder analysieren	Zielgruppen beschreiben und Bedürfnisse ableiten	Wo 1	Wo 1	4	
1.2.2	User Stories formulieren	Funktionale Anforderungen als User Stories erfassen	Wo 1	Wo 1	5	
1.2.3	Nicht-funktionale Anforderungen	Qualitätsanforderungen definieren und messbar machen	Wo 1	Wo 2	3	
1.2.4	Use-Case-Diagramm erstellen	Hauptprozesse modellieren und mit Akteuren verknüpfen	Wo 2	Wo 2	3	
1.3	Spezifikation	Datenmodell, Prozesse, Regeln, GUI-Skizzen ausarbeiten	Wo 2	Wo 2	17	M3
1.3.1	Datenmodell & Geschäftsobjekte	zentrale Objekte beschreiben und Beziehungen definieren	Wo 2	Wo 2	6	
1.3.2	Geschäftsprozesse & Aktivitätsdiagramm	Prozesse textlich und mit Aktivitätsdiagramm spezifizieren	Wo 2	Wo 2	5	
1.3.3	Geschäftsregeln & GUI-Skizzen	Regeln und erste Oberflächenentwürfe ausarbeiten	Wo 2	Wo 2	6	
2	Entwurf und Implementierung	Gem. Aufgabenstellung	Wo 3	Wo 6	77-87	M4-M6
2.0	Überarbeitung Phase-1	Tutor-Feedback in Projektdokumentation, Anforderungs- und Spezifikationsdokument einarbeiten	Wo 3	Wo 3	5	
2.1	Architektur-Design	MVVM/MVC, Klassendiagramm,	Wo 3	Wo 3	12	M4

		Sequenzdiagramm, Patterns planen				
2.1.1	Architekturentscheidungen treffen	MVVM/MVC, Schichten, Persistenz, zentrale Patterns festlegen	Wo 3	Wo 3	6	
2.1.2	UML-Klassendiagramm & Sequenzdiagramm	Hauptklassen und einen Kernablauf modellieren	Wo 3	Wo 3	6	
2.2	Basis-Implementierung	Kernlogik, Persistenz, GUI-Grundlayout, erste Tests	Wo 3	Wo 4	40-45	M5
2.2.1	Game-Logic implementieren	Spielregeln, Validierung, Scoring umsetzen	Wo 3	Wo 4	20-25	
2.2.2	GUI-Grundlayout & Interaktionen	Hauptfenster, Spielfeld, Basis-Interaktionen	Wo 4	Wo 5	15-20	
2.2.3	Persistenzschicht implementieren	SQLite-Anbindung, Speichern/Laden von Spielständen	Wo 5	Wo 5	5-10	
2.3	Erweiterungen & Refactoring	zusätzliche Features, Aufräumen des Codes, Doku-Abgleich	Wo 5	Wo 6	20-25	M6
2.3.1	Refactoring & Code-Aufräumen	Strukturverbesserungen, SOLID-Anpassungen, technische Schulden reduzieren	Wo 5	Wo 5	10-15	
2.3.2	Doku-Abgleich & Architektur-Update	Implementierung mit Doku/Architektur synchronisieren	Wo 6	Wo 6	10-15	
3	Test und Abschluss	Gem. Aufgabenstellung	Wo 6	Wo 8	45	M7-M8
3.0	Überarbeitung Phase-2	Tutor-Feedback in einarbeiten	Wo 6	Wo 6	5	
3.1	Testdesign und Testdurchführung	Testfälle definieren, automatisierte Tests, manuelle Tests	Wo 6	Wo 7	18	M7
3.1.1	Testfälle entwerfen	Teststrategie und Testfälle (inkl. TDD-Ziele) definieren	Wo 6	Wo 6	8	
3.1.2	Tests umsetzen & ausführen	automatisierte und manuelle Tests implementieren und durchführen	Wo 7	Wo 7	10	
3.2	Projektabschluss & Dokumente	Benutzeranleitung, Abstract, finale Doku & Abgabe vorbereiten	Wo 7	Wo 8	22	M8

3.2.1	Benutzeranleitung & Projektdoku final	Installations-/Nutzungsanleitung und finale Projektdokumentation	Wo 7	Wo 7	12	
3.2.2	Abstract & Abgabevorbereitung	Abstract schreiben, GitHub/ZIP prüfen, finale Abgabe vorbereiten	Wo 8	Wo 8	10	

Quelle: Eigene Darstellung

4.3 Detailbeschreibung und Meilensteine

Der Plan basiert auf der Verwendung von Kalenderwochen (z. B. Woche 1, Woche 2), anstelle von festen Terminen. Dies ermöglicht eine flexible Anpassung des Zeitplans. Dies ist notwendig da die Zeit für das Tutor-Feedback nicht bekannt ist und den Zeitplan stark ändern könnte. Im Zeitplan sind signifikante Meilensteine definiert, zwischen denen bewusst Pufferzeiten eingeplant wurden. Diese Puffer dienen dazu, notwendige Überarbeitungen und Anpassungen vorzunehmen, ohne den Gesamtzeitrahmen des Projekts zu gefährden. Diese Meilensteine dienen als Orientierungspunkte, wobei etwaige Abweichungen im Abstract reflektiert werden.

Die Meilensteine sind wie folgt festgelegt:

Phase 1: Planung und Anforderungsanalyse (Wochen 1-2, 40 Std.)

- 1.1. Projektplanung (8 Std.)
 M1: Projektplan fertig (Ende Woche 1)
- 1.2. Anforderungsermittlung (15 Std.)
 M2: Anforderungen dokumentiert (Mitte Woche 2)
- 1.3. Spezifikation (17 Std.)
 M3: Phase 1 Abgabe PebblePad (Ende Woche 2)

Phase 2: Entwurf und Implementierung (Wochen 3-6, 80 Std.)

- 2.1. Architektur/Design (12 Std.)
 M4: Architektur festgelegt+GUI-Risiko-Review (Ende Woche 3)
- 2.2. Implementierung (68 Std.)
 M5: Basis-Implementierung läuft (Ende Woche 4)

🔑 M6: Phase 2 Abgabe PebblePad + GitHub (Ende Woche 6)

Phase 3: Test und Abschluss (Wochen 7-8, 40 Std.)

3.1. Testen (18 Std.):

🔑 M7: Tests abgeschlossen + Bugfixes (Mitte Woche 7)

3.2. Projektabschluss (22 Std.):

🔑 M8: Finale Abgabe (GitHub-ZIP + PebblePad, Ende Woche 8)

4.4 Plan-Ist-Vergleich der Zeitplanung

In diesem Kapitel werden die geplanten Aufwände aus den Arbeitspaketen (Kapitel 4.2) den tatsächlich angefallenen Zeiten gegenübergestellt. Ziel ist es, Abweichungen je Phase bzw. Arbeitspaket zu identifizieren und deren Ursachen zu reflektieren, um Schlussfolgerungen für zukünftige Projekte abzuleiten.

Tabelle 3: Plan-Ist-Vergleich

PSP-Code	Arbeitspaket	Geplanter Aufwand (h)	IST- Aufwand (h)	Kommentar/Begründung
1.1	Projektplanung	8	10	Der Aufwand für die Projektplanung fiel höher aus, weil zunächst mehrere mögliche Projektideen und Technologie-Stacks bewertet wurden, bevor die Entscheidung für „Network Architect“ und PySide6 getroffen wurde. Zusätzlich benötigte die Risikobewertung mehr Zeit.
1.1.1	Projektziel & Scope definieren	3	3	
1.1.2	Risikoliste erstellen	3	4	
1.1.3	Vorgehensmodell & Phasen festlegen	2	3	
1.2	Anforderungs-ermittlung	15	14	Die Anforderungsermittlung benötigte mehr Zeit als geplant, da während des Schreibens neue funktionale Ideen und Varianten der Spielmechanik entstanden, die jeweils auf Konsistenz mit dem ursprünglichen Scope geprüft werden mussten.
1.2.1	Stakeholder analysieren	4	4	
1.2.2	User Stories formulieren	5	5	
1.2.3	Nicht-funktionale Anforderungen	3	3	
1.2.4	Use-Case-Diagramm erstellen	3	3	
1.3	Spezifikation	17	15	Bei der Spezifikation entstand Mehrarbeit, weil das Datenmodell und die Geschäftsprozesse in mehreren Iterationen verfeinert wurden, um alle relevanten Spielzustände und Sonderfälle (z. B. Netzwerk-Performance, Redundanz) abzudecken.
1.3.1	Datenmodell & Geschäftsobjekte	6	6	
1.3.2	Geschäftsprozesse & Aktivitätsdiagramm	5	5	
1.3.3	Geschäftsregeln & GUI-Skizzen	6	4	
2.0	Überarbeitung Phase-1	5	7	Überarbeitung Phase-1 dauerte länger als geplant – das Tutor-Feedback war umfangreich und erforderte tiefgreifende

				Anpassungen der Gantt-Diagramme und des Inhalts.
2.1	Architektur-Design	12	12	Architektur-Design verlief planmäßig. MVVM-Struktur, Klassendiagramm und Sequenzdiagramm konnten innerhalb des geplanten Zeitrahmens erarbeitet und im Architekturdokument festgehalten werden
2.1.1	Architekturentscheidungen treffen	6	+	
2.1.2	UML-Klassendiagramm & Sequenzdiagramm	6	6	
2.2	Basis-Implementierung	40-45	85	Größte Abweichung der Phase – Risiken R4 und R5 eingetreten; Game-Logic und GUI deutlich aufwändiger als geplant
2.2.1	Game-Logic implementieren	20-25	52	
2.2.2	GUI-Grundlayout & Interaktionen	15-20	29	
2.2.3	Persistenzschicht implementieren	5-10	3	
2.3	Erweiterungen & Refactoring	20-25	10	Verlief deutlich schneller als geplant, da die saubere Implementierung in Phase 2.2 wenig technische Schulden hinterließ und der Code bereits gut strukturiert war.
2.3.1	Refactoring & Code-Aufräumen	10-15	4	
2.3.2	Doku-Abgleich & Architektur-Update	10-15	6,5	
3.0	Überarbeitung Phase-2	5	5	Überarbeitung Phase-2 verlief planmäßig.
3.1	Testdesign und Testdurchführung	18	12	Testdesign und Testdurchführung verliefen effizienter als geplant. Da Unit- und Integrationstests bereits während der Implementierung vorbereitet und umgesetzt wurden, war in Phase 3 vor allem die Dokumentation im Testdokument sowie eine leichte Überarbeitung erforderlich, wodurch der tatsächliche Aufwand unter der ursprünglichen Planung blieb.
3.1.1	Testfälle entwerfen	8	3	
3.1.2	Tests umsetzen & ausführen	10	10	
3.2	Projektabschluss & Dokumente	22	10	Projektabschluss und Dokumentation verliefen effizienter als geplant. Dank
3.2.1	Benutzeranleitung & Projektdoku final	12	5	

3.2.2	Abstract & Abgabevorbereitung	10	5	laufend gepflegter Projektdokumentation und klarer Ordnerstruktur konnten Benutzeranleitung, Abstract und Abgabe deutlich schneller erstellt werden als vorgesehen.
-------	-------------------------------	----	---	---

Quelle: Eigene Darstellung

Phase 1 verlief insgesamt planmäßig und konnte innerhalb des vorgesehenen Zeitrahmens von zwei Wochen abgeschlossen werden. Die Arbeitspakete Projektplanung, Anforderungsermittlung und Spezifikation wurden sequenziell erarbeitet und in den entsprechenden Dokumenten festgehalten. Lediglich die Erstellung der Gantt-Diagramme erwies sich als zeitintensiver als erwartet, was jedoch durch paralleles Arbeiten an anderen Paketen ausgeglichen werden konnte. Insgesamt bildete die solide Planungsphase eine gute Grundlage für die nachfolgende Implementierung

Phase 2 wies die deutlichsten Abweichungen vom ursprünglichen Zeitplan auf. Das Architektur-Design verlief noch planmäßig und konnte mit der Ausarbeitung von MVVM-Struktur, Klassendiagramm und Sequenzdiagramm fristgerecht abgeschlossen werden. Anders verhielt es sich bei der Basis-Implementierung: Die Umsetzung der Game-Logic sowie des GUI-Grundlayouts überstieg die geplanten 40–45 Stunden mit rund 85 tatsächlich geleisteten Stunden erheblich. Ursächlich hierfür waren die unterschätzte Komplexität der Validierungslogik, der BridgeType-Implementierung sowie der QGraphicsView-basierten Benutzeroberfläche – womit sich die im Risikomanagement identifizierten Risiken R4 (UI-Komplexität) und R5 (Learning Curve) konkret bestätigten. Das abschließende Paket Erweiterungen & Refactoring konnte hingegen deutlich schneller als geplant abgeschlossen werden, da die konsequent saubere Implementierung kaum technische Schulden hinterlassen hatte und der Code bereits gut strukturiert war. In Summe war Phase 2 die zeitlich anspruchsvollste Phase des gesamten Projekts.

Phase 3 verlief insgesamt effizienter als ursprünglich geplant und konnte im vorgesehenen Zeitrahmen abgeschlossen werden. Die Arbeitspakete Überarbeitung Phase-2, Testdesign und Testdurchführung sowie Projektabschluss und Dokumente wurden sequenziell bearbeitet und in Testdokument, Projektdokumentation und Benutzeranleitung festgehalten. Durch die frühzeitige Implementierung von Unit- und Integrationstests während der Programmierung sowie die laufend gepflegte

Dokumentation und eine klare Ordnerstruktur reduzierte sich in der Finalisierungsphase der zusätzliche Aufwand, sodass Testdokument, Benutzeranleitung und Abgabeunterlagen mit weniger Nacharbeit als erwartet fertiggestellt werden konnten. Insgesamt bildete die strukturierte Finalisierungsphase eine stabile Grundlage für die bewertbare Abgabe von „Network Architect“ inklusive Tests, Dokumentation und lauffähiger Anwendung.

4.5 Reflexion der Zeitplanung und Abweichungen

Im Verlauf der Implementierung konnten zwei funktionale Anforderungen aufgrund des begrenzten Zeitrahmens nicht vollständig umgesetzt werden. Die User Story „Als Puzzle-Enthusiast möchte ich Hinweise ein-/ausschalten“ sowie die User Story „Als IT-Student möchte ich Tutorials zu Netzwerk-Konzepten sehen“ wurden bewusst als technische Schuld zurückgestellt, da die Kernfunktionalität der Anwendung priorisiert wurde. Beide Anforderungen sind im Anforderungsdokument entsprechend gekennzeichnet und stellen potenzielle Erweiterungen für zukünftige Versionen von „Network Architect“ dar.

Darüber hinaus zeigte sich in der Finalisierungsphase eine weitere technische Einschränkung bei der Prüfung großer Level. Mit steigender Anzahl an Knoten und gelegten Bridges erhöht sich die Zeit für die Validierung einer Lösung merklich. Diese Problematik wurde im Rahmen von Phase 3 erkannt und als bewusst akzeptierte technische Schuld dokumentiert, da eine grundlegende algorithmische Optimierung den verfügbaren Zeitrahmen überschritten hätte. Für die Abgabe wurde daher die funktionale Korrektheit der Lösungsprüfung priorisiert, während die Performance bei großen Levels als Einschränkung in Kauf genommen wurde.

5 Benutzeranleitung

5.1 Installation und Start

Zur Nutzung von „Network Architect“ wird ein Computer mit Windows 10 oder Windows 11 benötigt. Die Anwendung wird als ausführbare Datei bereitgestellt und kann ohne separate Installation von Python verwendet werden.

Zum Start lädt der Benutzer die Datei Network Architect.exe herunter und öffnet sie per Doppelklick. Eine zusätzliche Installation ist nicht erforderlich. Nach dem Start erscheint das Hauptmenü der Anwendung mit den drei Schaltflächen New Game, Statistic und Exit.

Über New Game gelangt der Benutzer zur Levelauswahl. Dort kann ein vorhandener Spieler ausgewählt oder über + New Player ein neuer Spieler angelegt werden. Ist beim ersten Start noch kein Spieler vorhanden oder wurde die Benutzerdatenbank entfernt, weist die Anwendung zunächst darauf hin, dass zuerst ein neuer Spieler angelegt werden muss.

Über Statistic können bereits gespeicherte Spieler- und Levelstatistiken angezeigt werden. Mit Exit wird die Anwendung beendet.

Falls beim ersten Start eine Sicherheitsabfrage von Windows erscheint, muss die Ausführung der Anwendung bestätigt werden. Sollte das Programm nicht starten, ist zunächst zu prüfen, ob ein unterstütztes Windows-System verwendet wird und ob der Download der .exe vollständig und unverändert erfolgt ist.

Beim ersten Start legt „Network Architect“ automatisch eine lokale Datenbankdatei user_DB.db im Benutzerprofil an. Standardmäßig befindet sich diese im Ordner C:\Users\<Benutzername>\AppData\Local\NetworkArchitect.

In dieser Datei werden die angelegten Spieler sowie der Spielfortschritt (freigeschaltete Level und Statistikwerte) gespeichert. Wird die Datei gelöscht, gehen die gespeicherten Spieler und Fortschritte verloren und beim nächsten Start muss wieder ein neuer Spieler angelegt werden.

5.2 Kurzanleitung

Nach dem Start der Anwendung erscheint das Hauptmenü mit den Schaltflächen New Game, Statistic und Exit. Über New Game gelangt der Benutzer zur Levelauswahl. Dort wird zunächst ein vorhandener Spieler aus dem Dropdown-Menü ausgewählt oder über + New Player ein neuer Spieler angelegt. Für neue Spielernamen sind 2 bis 20 Zeichen zulässig. Anschließend kann eines der freigeschalteten Level aus der Liste ausgewählt und über Start Game gestartet werden.

Im Spielbildschirm befindet sich in der Mitte das jeweilige GameBoard. Links stehen die Bedienfunktionen Hauptmenü, Levelauswahl, Reset, Lösung prüfen und Hilfe zur Verfügung. Rechts werden das aktuell verfügbare Budget sowie die auswählbaren Verbindungstypen angezeigt. Zur Auswahl stehen ETHERNET mit 100 Mbps und Kosten von 10€, WIFI_24G mit 300 Mbps und 20€, WIFI_5G mit 600 Mbps und 35€ sowie FIBER mit 2000 Mbps und 120€.

Ziel des Spiels ist es, alle Netzknoten mit dem Server zu verbinden und dabei ein gültiges Netzwerk innerhalb des verfügbaren Budgets aufzubauen. Jeder Level enthält genau einen Server (S) als zentralen Ausgangspunkt des Netzwerks. Zusätzlich können Client (C), Router (R) und Firewall (F) vorkommen. Der Server benötigt mindestens zwei

Verbindungen, Firewalls dürfen maximal zwei Verbindungen besitzen und alle übrigen Knoten maximal vier Verbindungen.

Zum Platzieren einer Verbindung wird zuerst rechts ein Verbindungstyp ausgewählt. Anschließend klickt der Benutzer den ersten Knoten, danach die gewünschten GridPoints des Pfads und schließlich den Zielknoten an. Mit Brücke platzieren wird die Verbindung erstellt. Mit Brücke löschen können vorhandene Verbindungen wieder entfernt werden. Ein Rechtsklick setzt eine aktuelle Auswahl zurück. Wird ein Level über Reset neu gestartet, werden sowohl die bisher platzierten Verbindungen als auch die Zeit des aktuellen Versuchs zurückgesetzt.

Ein Level gilt als erfolgreich abgeschlossen, wenn alle Knoten regelkonform mit dem Server verbunden sind. Über Lösung prüfen wird der aktuelle Stand ausgewertet. Nach erfolgreichem Abschluss zeigt die Anwendung einen Ergebnisdiallog mit Performance, Redundanz und Zeit an. Redundanz gibt an, wie viele Verbindungen entfernt werden könnten, ohne dass das Level seine Gültigkeit verliert. Performance bewertet die Qualität der Verbindung aller Clients zum Server anhand der jeweils besten verfügbaren Pfade. Über Nächstes Level startet direkt das folgende Level, über Weiter spielen kann das aktuelle Level weiter optimiert werden, solange noch Budget verfügbar ist, und über Hauptmenü wird zum Hauptmenü zurückgekehrt.

Über Statistic im Hauptmenü können gespeicherte Ergebnisse eingesehen werden. In der Statistikansicht gibt es die Tabs Player Statistics und Level Statistics. Im ersten Tab wird über ein Dropdown ein Spieler ausgewählt und dessen Ergebnisse werden tabellarisch angezeigt. Im zweiten Tab wird ein Level ausgewählt, woraufhin die gespeicherten Ergebnisse der Spieler zu diesem Level angezeigt werden. Mit Back to Menu gelangt der Benutzer zurück ins Hauptmenü.

Bei bestimmten Bedienfehlern gibt die Anwendung eine Meldung aus. Wird auf dem GameBoard gearbeitet, ohne zuvor einen Verbindungstyp auszuwählen, erscheint ein entsprechender Hinweis. Wird zuerst ein GridPoint statt eines Knotens ausgewählt oder ein ungültiger Pfad aufgebaut, wird dies ebenfalls durch eine Fehlermeldung angezeigt. Beim Anlegen eines neuen Spielers werden außerdem leere, zu kurze, zu lange oder bereits vorhandene Namen zurückgewiesen. Beim Wechsel ins Hauptmenü oder in die Levelauswahl während eines laufenden Levels erscheint zusätzlich eine Warnung, dass der aktuelle Fortschritt verloren geht.

6 Reflexion zentraler Entscheidungen und Alternativen

In diesem Kapitel werden zentrale Entscheidungen des Projekts Network Architect rückblickend reflektiert und in Beziehung zu den in Kapitel 1 formulierten Zielen, der Problemstellung sowie den Rahmenbedingungen der Prüfungsleistung gesetzt. Im Fokus stehen das gewählte Vorgehensmodell, die technologischen Grundlagen, die Architekturentscheidung für MVVM sowie ausgewählte Alternativen, die im Projektverlauf betrachtet, aber nicht umgesetzt wurden. Ziel dieser Reflexion ist es, Stärken und Schwächen der getroffenen Entscheidungen transparent zu machen und Ansatzpunkte für Verbesserungen in zukünftigen Projekten abzuleiten.

6.1 Reflexion des Vorgehensmodells

Für Network Architect wurde das iterative Wasserfallmodell gewählt, da es die in der Aufgabenstellung vorgegebene Phasenstruktur (Konzeption, Realisierung, Test und Abschluss) gut abbildet und gleichzeitig begrenzte Rückkopplungsschleifen zwischen den Phasen erlaubt. In der Praxis erwies sich diese Wahl als passend, weil die Artefakte aus Phase 1 (Anforderungen, Spezifikation) tatsächlich als Referenz für die Implementierung und das Testen dienten und bei Bedarf iterativ nachgeschärft wurden, etwa beim Datenmodell und den Geschäftsregeln.

Für ein größeres Teamprojekt mit mehreren Entwicklern oder echten Endnutzern wäre ein stärker inkrementelles Vorgehen, beispielsweise Scrum mit kurzen Sprints und regelmäßigen Review-Terminen, voraussichtlich besser geeignet, um frühe Prototypen und Nutzerfeedback enger in die Entwicklung einzubinden. Im Kontext dieses Einzelprojekts mit klaren Abgabeterminen und begrenzter Kommunikationskomplexität war das iterative Wasserfallmodell dennoch eine pragmatische und angemessene Wahl, die Struktur und Planbarkeit sichergestellt hat.

6.2 Reflexion technologischer Entscheidungen

Die technologische Basis von Network Architect mit Python als Programmiersprache, PySide6 als GUI-Framework, SQLite als Persistenzlösung und PyInstaller für das Deployment wurde einerseits durch die Vorgaben der Prüfungsleistung, andererseits durch persönliche Vorerfahrung und verfügbare Lernressourcen geprägt. Python und PySide6 ermöglichten eine vergleichsweise schnelle Umsetzung einer GUI-basierten Desktop-Anwendung mit QGraphicsView-Unterstützung, was für die Visualisierung von Nodes und Bridges im Spielfeld entscheidend war. SQLite bot eine leichtgewichtige, in

Python integrierte Datenbank, mit der sich Spielstände und Statistiken ohne zusätzliche Server-Infrastruktur speichern lassen.

Im Projektverlauf zeigte sich, dass diese Entscheidungen zwar den Einstieg erleichterten, aber auch spezifische Herausforderungen mit sich brachten. Insbesondere erwies sich QGraphicsView als leistungsfähig, aber komplex in der Handhabung, sodass ein signifikanter Teil der Lernkurve auf GUI-Technik statt auf fachliche Spielmechanik entfiel.

Alternativ wäre eine Web-basierte Umsetzung mit HTML/JavaScript-Frontend und einem einfachen Backend denkbar gewesen, die das Deployment an Endnutzer vereinfacht und prinzipiell plattformunabhängigen Zugriff ermöglicht hätte. Diese Variante hätte jedoch zusätzliche Technologien, Build-Pipelines und Hosting-Fragen eingeführt, die den Rahmen dieses Einzelprojekts voraussichtlich überschritten hätten. Ebenso hätte eine rein dateibasierte Persistenz mit JSON- oder YAML-Dateien die Datenbankebene vereinfacht, jedoch auf Komfortfunktionen und Integritätsmechanismen von SQLite verzichtet. Insgesamt waren die getroffenen Technologieentscheidungen für das gegebene Zielbild und den Projektumfang geeignet, auch wenn sie an einzelnen Stellen zu erhöhtem technischen Risiko und Mehraufwand führten.

6.3 Reflexion der Architekturentscheidung

Die Entscheidung für eine MVVM-Architektur zielte darauf ab, die grafische Oberfläche klar von Spiel- und Netzwerklogik zu trennen und damit Testbarkeit, Wartbarkeit und Erweiterbarkeit der Anwendung zu verbessern. In der Umsetzung von Network Architect hat sich dieser Ansatz grundsätzlich bewährt: Zentrale Geschäftsregeln wie Budgetprüfung, Topologie-Validierung und Redundanzbewertung konnten im Model-Layer implementiert und über ViewModels in die GUI eingebunden werden, ohne dass die Views selbst komplexe Logik enthalten mussten. Dies erleichterte das schrittweise Refactoring und die Synchronisation mit den im Architekturdokument beschriebenen Klassen.

Gleichzeitig zeigte sich, dass die konsequente Anwendung von MVVM in Kombination mit PySide6 zusätzlichen Strukturierungs- und Koordinationsaufwand erzeugt. Insbesondere das saubere Signal- und Property-Binding zwischen View und ViewModel sowie die Verantwortungstrennung zwischen GameViewModel und den Model-Klassen erforderten mehrere Iterationen, um eine klare und konsistente Schnittstellengestaltung zu erreichen. Für ein kleineres Puzzle-Projekt mit weniger anspruchsvoller GUI wäre eine vereinfachte Schichtenarchitektur oder ein klassisches MVC-Muster möglicherweise schneller umzusetzen gewesen, wenn auch mit geringerer Testbarkeit der Logik ohne GUI.

Rückblickend war die Wahl von MVVM für dieses Lernprojekt dennoch sinnvoll, da sie eine für Desktop-Anwendungen typische Architektur einführt und direkte Bezüge zu den Aufgabenstellungen in den Bereichen UML-Modellierung und Testbarkeit herstellt. Für zukünftige Projekte erscheint es jedoch sinnvoll, frühzeitig Hilfsklassen und Services einzuplanen, um zu große ViewModels zu vermeiden und die Komplexität einzelner Komponenten zu begrenzen.

6.4 Betrachtete Alternativen

Im Rahmen des Projekts wurden mehrere alternative Ansätze für Vorgehensmodell, Technologie-Stack und Architektur diskutiert, letztlich aber nicht umgesetzt. Die folgende Übersicht fasst zentrale Alternativen zusammen und stellt die wesentlichen Gründe für ihre Nicht-Auswahl dar.

Tabelle 4: Betrachtete Alternativen

Entscheidungs- bereich	Gewählte Option	Alternative	Vorteile der Alternative	Gründe gegen die Alternative
Vorgehens- modell	Iteratives Wasserfall modell	Scrum / agiles Vorgehen	Häufiges Feedback, frühere UI-Validierung, flexible Prioritäten	Höherer organisatorischer Overhead, kein echtes Team, feste Abgaben
Technologie- Stack	Python + PySide6 + SQLite	Web-App (JS-Fronten d + API)	Plattformunabhäng iger Zugriff, einfacheres Deployment für Nutzer	Zusätzliche Technologien und Infrastruktur, höherer Lern- und Setup-Aufwand
Persistenz	SQLite-Dat enbank	JSON/YAML -Dateien	Geringere technische Komplexität, keine DB-Konfiguration	Schwächere Integrität, unkomfortable Abfragen, schwieriger für Statistiken
Architektur	MVVM (Desktop- Architektu r)	Einfaches MVC / Schichtenm odell	Weniger Boilerplate, schnellere Umsetzung für kleine Projekte	Geringere Testbarkeit der Logik ohne GUI, schwächere Trennung von View und Logik

Quelle: Eigene Darstellung

Diese Alternativen zeigen, dass in allen Bereichen bewusst Entscheidungen zugunsten einer stärkeren strukturellen und methodischen Ausrichtung (z.B. MVVM, SQLite, iteratives Wasserfallmodell) getroffen wurden, auch wenn dadurch teilweise höhere Einstiegshürden und Mehraufwände in Kauf genommen wurden. Für ein größeres Folgeprojekt mit mehr Zeit und einem Team wäre insbesondere eine agilere Vorgehensweise in Kombination mit einem Web-basierten Frontend eine naheliegende Weiterentwicklung, um Nutzerfeedback und Plattformunabhängigkeit stärker in den Fokus zu rücken.

7 Zusammenfassung und Ausblick

Network Architect setzt das klassische Hashiwokakero-Spielprinzip in eine moderne Netzwerk-Thematik um und kombiniert damit logische Rätsel mit grundlegenden Konzepten der IT-Infrastruktur. Im Projekt wurde eine funktionsfähige Windows-Desktop-Anwendung mit MVVM-Architektur, QGraphicsView-basierter Spielfläche und SQLite-Persistenz realisiert, die es Puzzle-Enthusiasten und IT-Studierenden ermöglicht, Netzwerk-Topologien zu entwerfen und hinsichtlich Performance und Redundanz zu bewerten. Die enge Kopplung von Anforderungen, Spezifikation, Architektur und Implementierung zeigt, dass klassische Software-Engineering-Methoden auch bei einem edukativen Puzzle-Spiel sinnvoll angewendet werden können.

Gleichzeitig macht das Projekt – insbesondere in der Problemstellung (Kapitel 1.2) und den Limitationen (Kapitel 1.6) – deutlich, dass die realisierten Funktionen nur einen Ausschnitt der möglichen fachlichen und technischen Tiefe abbilden. Die Netzwerk-Simulation bleibt bewusst vereinfacht, die Zielplattform auf Windows 10/11 begrenzt und Aspekte wie Usability-Tests mit realen Nutzern, Barrierefreiheit oder Mehrbenutzer-Szenarien wurden im Rahmen des Portfolio-Umfangs nicht umgesetzt. Die Reflexion in Kapitel 5 zeigt zudem, dass die gewählten Entscheidungen (iteratives Wasserfallmodell, Python/PySide6, MVVM, SQLite) zwar angemessen waren, aber jeweils auch Alternativen mit anderen Stärken und Schwächen existieren.

Für eine mögliche Weiterentwicklung von Network Architect ergeben sich daraus mehrere konkrete Ansatzpunkte. Fachlich könnten realitätsnähere Netzwerk-Features, zusätzliche Level-Typen oder dynamische Störungen (z.B. Ausfälle wichtiger Nodes) eingeführt werden, um die Lernkomponente zu vertiefen. Technisch wären eine Ausweitung der Plattformunterstützung, eine Web-basierte Variante sowie eine systematischere Testabdeckung der GUI und Sicherheitsaspekte sinnvolle Schritte. Methodisch würden frühzeitige Prototypen, formale Usability-Tests mit Vertretern der Zielgruppen und ein agileres Vorgehensmodell helfen, Benutzerfeedback stärker in den Entwicklungsprozess einzubinden.

Aus Software-Engineering-Sicht zeigt das Projekt, dass eine klare Problemformulierung, strukturierte Anforderungen, ein konsistentes Architekturkonzept und bewusste Risiko-Betrachtung die Umsetzung auch eines relativ kleinen Einzelprojekts deutlich unterstützen. Die im Projektverlauf gewonnenen Erkenntnisse zu Technologie-Wahl, Architektur-Trade-offs und Zeitplanung liefern eine Basis, um zukünftige Projekte –

sowohl im Spiele- als auch im Anwendungsbereich – gezielter zu planen und von Beginn an stärker auf Testbarkeit, Nutzerbeteiligung und Erweiterbarkeit auszurichten.

8 Verzeichnisse

8.1 Tabellenverzeichnis

Tabelle 1: Risikoübersicht	12
Tabelle 2: Arbeitspakete	20
Tabelle 3: Plan-Ist-Vergleich	25
Tabelle 4: Betrachtete Alternativen	34

8.2 Abbildungsverzeichnis

Abbildung 1: Iteratives Wasserfall-Diagramm	10
Abbildung 2: Gantt-Diagramm Phase 1	19
Abbildung 3: Gantt-Diagramm Phase 2	20
Abbildung 4: Gantt-Diagramm Phase 3	20

8.3 Abkürzungsverzeichnis

GUI – Graphical User Interface

IT – Informationstechnologie

LAN – Local Area Network

WAN – Wide Area Network

MVC – Model View Controller

MVVM – Model View ViewModel

NFR – Nichtfunktionale Anforderungen

UML – Unified Modeling Language